from case study of matrix testing using Junit framework.

Matrix Testing is a type of unit testing used to verify mathematical operations performed on matrices such as

* Addition.
* Substraction.
* multiplication.
* Transpose.
* Determinant.

Junit framework is widely used for testing Java applications.

Objectives.

The objective is to:

* Validate correction of matrix operations.
* Detect logical errors.
* ~~Detect t~~
* Automate testing process

Problem Statement

Develop a Java class matrix operation that performs:

1. matrix Addition
2. matrix multiplication
3. matrix Transpose

Then create Junit test cases to verify correction.

Implementation

matrix Operation.java

```
public class matrixOperations
{
    public static int[][] add (int[][] A, int[][]B)
    {
        int rows = A.length;
        int cols = A[0].length;
        int[][] result = new int [rows][cols];
        for (int i=0; < rows; i++)
        {
            for (int j=0; j<cols; j++)
            {
            result [i][j] = A[i][j] + B[i][j];
            }
        }
        return result;
    }

    // similarly multiply logic.
    // transpose logic & so on...
```

Test Case

matrixOperationsTest.java

```
import static org.junit.jupiter.api.Assertions.*
import org.junit.jupiter.api.Test;
public class matrixOperationTest {
    @Test
    public void testAddition ()
    {
        int[][]A = { {1,2}, {3,6} };
        int[][] B = { {5,6}, {7,8} };
```

Execution

when executed using Junit:

- All test cases pass successfully.
- Any incorrect logic will produce assertion failures
- Automated testing reduces manual verification effort

Testing scenarios covered

1. Normal 2x2 matrices
2. Positive Integers
3. Correct dimensional compatibility.
4. Output matrix validation.

Additional tests can include.

- Null matrices
- Incompatible dimension
- Large matrices
- Negative numbers.

Advantages

→ Automated test execution.
→ Immediate feedback.
→ Reusable testcases.
→ Supports regression testing.
→ easy integration with IDEs